

IMPLEMENTATION

Matrix Eigenvalues and Google's PageRank Algorithm

1. Software Architecture

The project is implemented in Python using a modular structure. The implementation separates matrix/eigenvalue operations, graph construction, the eigenvector-based PageRank method, and the power-iteration method. This makes the mathematical steps independently testable and keeps the PageRank pipeline reproducible.

Module	Responsibility
eigenvalues.py	Eigenvalues/eigenvectors, eigenvalue-1 selection, and probability-vector normalization.
pagerank.py	Transition matrix, Google matrix, eigenvector PageRank, and fixed-point verification.
power_iteration.py	Transition/Google matrix construction and iterative PageRank computation.
graph.py	Sample graphs, adjacency matrices, graph/PageRank visualization, and convergence plots.
pagerank_development.ipynb	Experimental workflow, graph experiments, numerical comparisons, and visual analysis.
test_pagerank.py	Automated tests for normalization, dangling nodes, and the PageRank fixed-point relationship.

Table 1. Main software components used in the implementation.

2. Graph and Adjacency Matrix

A directed graph represents a small web network. Each node represents a page and each directed edge represents a hyperlink. The project uses the convention $A[i,j] = 1$ when page i links to page j . This row-based adjacency representation is converted into a column-stochastic transition matrix.

3. Transition Matrix

For PageRank computation, M is column-stochastic. The entry $M[\text{target}, \text{source}]$ represents the probability of moving from a source page to a target page. If a source page has k outgoing links, each outgoing link receives probability $1/k$.

Dangling nodes: A page with no outgoing links receives a uniform transition column. For N pages, every entry in that column is $1/N$. This prevents probability mass from becoming trapped.

4. Google Matrix

The transition matrix is combined with teleportation using the damping factor. The main experiments use $d = 0.85$.

$$G = dM + (1 - d)(1/N)E$$

Here M is the transition matrix, E is an $N \times N$ matrix of ones, N is the number of pages, and d is the damping factor. The resulting Google matrix remains column-stochastic.

5. Eigenvector-Based PageRank

PageRank is formulated as an eigenvalue problem. The PageRank vector is the normalized eigenvector corresponding to eigenvalue 1 of the Google matrix.

$$G \cdot PR = PR$$

The implementation uses NumPy to calculate the eigenpairs, selects the eigenvalue closest to 1, extracts its corresponding eigenvector, and normalizes the vector so that its entries sum to 1. Because eigenvectors may have either sign, the sign is adjusted before normalization when necessary.

6. Power Iteration

A second implementation obtains PageRank iteratively. Starting from a uniform probability vector, the Google matrix is repeatedly applied.

$$\mathbf{PR}(k+1) = \mathbf{G} \cdot \mathbf{PR}(k)$$

The L1 difference between consecutive vectors is measured at each iteration. The algorithm stops when the difference falls below 10^{-4} , with a maximum of 1000 iterations.

7. Verification and Validation

Check	Condition
Probability normalization	$\sum \mathbf{PR} = 1$
Column stochasticity	$\sum \mathbf{G}_{ij} = 1$ for every column j
Fixed-point relation	$\ \mathbf{G} \cdot \mathbf{PR} - \mathbf{PR}\ $ is close to 0

Table 2. Numerical conditions used to validate the PageRank implementation.

8. Representative Result

For the four-page example graph with $d = 0.85$, the implementation produces the following PageRank values:

Page	PageRank
A	0.324561
B	0.175439
C	0.324561
D	0.175439

The PageRank values sum to approximately 1. The eigenvector calculation identifies an eigenvalue of approximately 1, and the resulting vector satisfies $\mathbf{G} \cdot \mathbf{PR} \approx \mathbf{PR}$ with numerical error close to machine precision for the direct eigenvector implementation.

9. Experimental Analysis

The implementation was evaluated on several graph structures, including dangling-node, spider-trap, hub, star, cycle, chain, and six-node graphs. The effect of the damping factor on power-iteration convergence was also investigated.

Damping factor	Iterations
0.50	17
0.70	22
0.85	27
0.95	31

Table 3. Power-iteration convergence for the representative graph.

The observed iteration count increases as the damping factor approaches 1. This indicates slower power-iteration convergence for larger damping factors in the tested graph.

10. Automated Testing

The project includes a pytest test suite covering transition-matrix normalization, dangling-node handling, PageRank normalization, and the fixed-point/eigenvector relationship.

```
python -m pytest
```

The current test run completes successfully with 4 passed tests.

Implementation Summary

The complete implementation converts a directed graph into a stochastic matrix, constructs the Google matrix, obtains PageRank both as the eigenvector associated with $\lambda = 1$ and through power iteration, and validates the results using numerical checks and automated tests.